

Overlapping Subproblems: From Recursion to Memoization

Programming and Data Structures — Week 9, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely what “overlapping subproblems” means and identify it directly within a concrete recursion tree; compute the exact exponential cost of naive recursive Fibonacci by counting function calls rather than merely citing a complexity class; implement memoization using a hash table from Week 8, and explain exactly why caching computed results eliminates redundant work; and state the two conditions — overlapping subproblems and optimal substructure — that a problem must satisfy before dynamic programming can be applied at all.

Outline

Today covers a redundant-homework analogy motivating the need to cache repeated work; naive recursive Fibonacci and its exact exponential cost; the recursion tree, showing precisely where redundant computation occurs; memoization, implemented directly using Week 8’s hash table; a proof, using the same recursion tree, of why memoization eliminates the redundancy; the two conditions — overlapping subproblems and optimal substructure — required before dynamic programming applies; the MTech-track precise derivation of the exponential blowup; an in-class quiz; and a summary.

The redundant-homework analogy

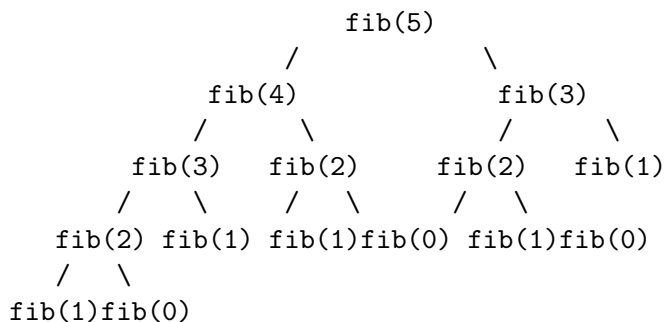
Imagine repeatedly solving the exact same textbook problem from scratch, twenty separate times, because each time you have forgotten that you already solved it. Each repetition costs the same full effort as the first attempt, since nothing from the previous solution is retained or reused. The obvious fix — write down the answer the first time, and check your notes before starting from scratch again — is the entire conceptual idea this lecture formalizes, applied to recursive function calls rather than homework problems.

Naive recursive Fibonacci

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

This is a direct, faithful translation of the mathematical Fibonacci definition into code, and it is entirely correct — it returns the right answer for every input. Yet computing `fib(30)` already takes a noticeably long time on a modern computer, and `fib(40)` takes dramatically, almost unreasonably longer, despite being asked for a result only 10 steps further along the sequence. Something is going very wrong with this implementation's efficiency, and identifying exactly what is the purpose of this lecture.

The recursion tree for fib(5)



Drawing out `fib(5)`'s complete recursion tree reveals the exact problem: `fib(3)` is computed twice, entirely independently, once as a child of `fib(4)` and once as a direct child of `fib(5)`. `fib(2)` is computed three separate times. `fib(1)` and `fib(0)` are recomputed even more often as the tree grows. This is "overlapping subproblems": the same smaller computation is needed repeatedly across different branches of the recursion, and naive recursion recomputes it from scratch every single time, with zero memory of having already done the work.

Counting the exact exponential cost

Let $T(n)$ denote the total number of function calls needed to compute `fib(n)` naively. Since computing `fib(n)` requires computing `fib(n-1)` and `fib(n-2)` plus one call for itself, $T(n) = T(n-1) + T(n-2) + 1$ — the identical recurrence shape as the Fibonacci sequence itself, meaning $T(n)$ grows at the same exponential rate as $F(n)$, which is $\Theta(\phi^n)$ where $\phi \approx 1.618$ is the golden ratio. Concretely, computing `fib(40)` this way requires roughly 331 million recursive calls — an enormous

avoidable cost for computing a single Fibonacci number, and the direct consequence of never reusing the overlapping subproblem results traced on the previous slide.

Memoization: caching with a hash table

```
def fib_memo(n, cache=None):
    if cache is None:
        cache = {}          # Week 8's hash table, via Python's built-in dict
    if n in cache:
        return cache[n]      # ALREADY COMPUTED - return instantly, no recursion at all
    if n <= 1:
        result = n
    else:
        result = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    cache[n] = result        # SAVE this result before returning
    return result
```

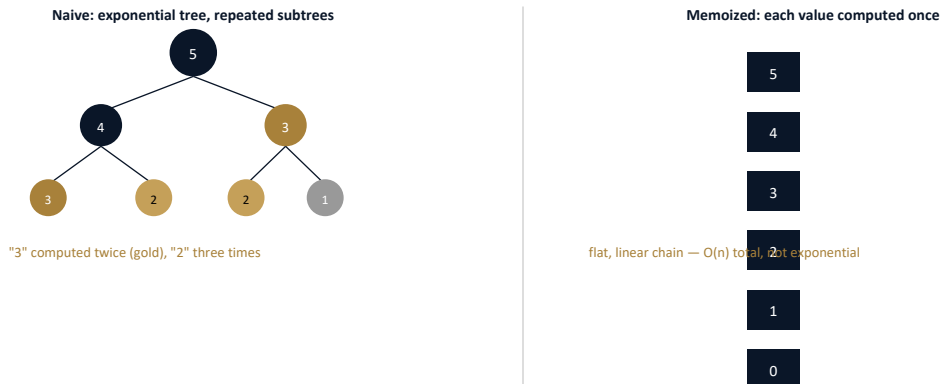
Memoization adds exactly two things to the naive implementation: a check at the start of every call, looking up whether this specific n has already been computed, and a save at the end, recording the result before returning. The cache itself is Week 8's hash table, accessed here via Python's built-in dict, giving the lookup and save each $O(1)$ average cost. This single addition is the entire fix — no other part of the recursive logic changes at all, and the function remains just as “obviously correct” as the naive version, since it still follows the exact same mathematical definition.

Tracing memoized `fib(5)`, showing the cache hits

```
fib_memo(5): not cached. needs fib(4), fib(3)
  fib_memo(4): not cached. needs fib(3), fib(2)
    fib_memo(3): not cached. needs fib(2), fib(1)
      fib_memo(2): not cached. needs fib(1), fib(0)
        fib_memo(1): base case, cache[1]=1
        fib_memo(0): base case, cache[0]=0
        cache[2] = 1+0 = 1
      fib_memo(1): CACHE HIT → returns 1 instantly, no recursion
    cache[3] = 1+1 = 2
  fib_memo(2): CACHE HIT → returns 1 instantly
  cache[4] = 2+1 = 3
  fib_memo(3): CACHE HIT → returns 2 instantly
  cache[5] = 3+2 = 5
```

Tracing the memoized version on the same `fib(5)` reveals exactly where the previous recursion tree's redundancy disappears: `fib(3)`, `fib(2)`, and `fib(1)` each get computed genuinely once, and every subsequent request for the same value is answered instantly from the cache, with no further recursive calls at all. Compare this directly against the earlier recursion tree diagram — every duplicated subtree there collapses into a single $O(1)$ cache lookup here.

Why memoization works: the recursion tree, collapsed



The naive recursion tree's exponential size comes entirely from the repeated subtrees rooted at the same value of n — memoization does not change the mathematical recurrence at all, it simply refuses to re-explore a subtree that has already been fully explored once, collapsing the tree's size from exponential down to linear in n .

The new cost: memoized Fibonacci is $O(n)$

With memoization, each distinct value from 0 up to the target n is computed exactly once, and each individual computation does only $O(1)$ work: one hash table lookup to check the cache, one addition, and one hash table write to save the result. Total cost is therefore $O(n) \times O(1) = O(n)$, compared to naive recursion's $\Theta(\phi^n)$ — for `fib(40)`, this is the difference between roughly 40 calls and roughly 331 million, a genuinely enormous improvement, not a marginal constant-factor tuning.

Measuring the improvement for real

```
import time

start = time.perf_counter()
fib(32)
```

```

naive_time = time.perf_counter() - start

start = time.perf_counter()
fib_memo(32)
memo_time = time.perf_counter() - start

print(f"naive: {naive_time:.4f}s    memoized: {memo_time:.6f}s")

```

Measuring both versions directly on `fib(32)` should show the naive version taking a noticeable fraction of a second or more, while the memoized version completes in a small fraction of a millisecond — a difference easily visible without any specialized profiling tools. This follows the same habit established in Week 2: measure a claimed complexity-class improvement directly on real code, rather than trusting the big-O argument alone, however rigorously it was derived.

The two conditions dynamic programming requires

Dynamic programming applies only to problems satisfying two conditions. Overlapping subproblems means the same smaller subproblem recurs multiple times across the natural recursive decomposition, exactly as `fib(3)` and `fib(2)` did in this lecture’s traced example. Optimal substructure means an optimal solution to the whole problem can be assembled directly from optimal solutions to its subproblems, with no need to reconsider choices already made at the subproblem level. Fibonacci satisfies both conditions almost trivially, which is precisely why it makes a clean first example — but the real skill this unit builds, starting next lecture, is recognizing both conditions in problems where they are far less obvious, such as optimization problems with genuine decision-making at each step.

MTech addition — proving the exponential blowup precisely

A rigorous, if slightly loose, lower bound can be proven by induction: $T(n) \geq 2^{n/2}$ for $n \geq 2$. The base cases $T(0)$ and $T(1)$ are both at least 1, satisfying $2^0 = 1$. The inductive step uses $T(n) = T(n-1) + T(n-2) + 1$, and since both $T(n-1)$ and $T(n-2)$ are each at least $2^{(n-2)/2}$ by the inductive hypothesis (using the weaker of the two available bounds for both terms), their sum already reaches $2 \cdot 2^{(n-2)/2} = 2^{n/2}$, before even adding the $+1$. This proves exponential growth rigorously using only elementary induction — the technique Week 7 Lecture 3 used to prove AVL tree height bounds — without needing the exact golden-ratio closed form to establish the qualitative conclusion that matters most: naive recursive Fibonacci is genuinely exponential, not merely “slow.”

A subtlety: memoization changes space cost too

It is worth being precise that memoization is a time-for-space trade, not a pure win with no cost. Naive recursion uses no explicit cache, but its call stack itself grows to depth n , exactly the $O(n)$ space cost

Week 3's call-stack analysis established for any recursive function. The memoized version adds an $O(n)$ hash table cache on top of that same $O(n)$ call-stack depth, meaning it genuinely uses more total memory than the naive version, not less. The trade being made is always time for space: an enormous, often decisive, time saving purchased with a real and quantifiable additional memory cost — worth stating explicitly rather than presenting memoization as a strictly free improvement.

Exercise

As an exercise, instrument both `fib` and `fib_memo` with a global call counter, incremented on every function entry, and confirm empirically that `fib_memo(30)` makes dramatically fewer calls than `fib(30)`, reporting the exact counts observed. Separately, reimplement `fib_memo` using Python's built-in `functools.lru_cache` decorator instead of a manually managed dictionary, and confirm it produces identical results — `lru_cache` is precisely this lecture's memoization pattern, provided as a reusable, general-purpose tool in the standard library.

Recognizing overlapping subproblems in the wild

A practical warning sign for overlapping subproblems: a recursive function being called with the exact same arguments more than once, across genuinely different call paths in the recursion tree, not merely as a coincidence of one specific input. Fibonacci's tell is structural: `fib(n-1)`'s own recursive expansion eventually calls `fib(n-2)` internally, which the outer `fib(n)` call already needed directly as its second term too — the same subproblem needed by multiple different “distances” up the recursion tree. Watching for this pattern — not just in Fibonacci, but in any recursive algorithm encountered from this point in the course onward — is the practical skill this lecture is building toward, ahead of considerably less obvious cases in the lectures that follow.

Quiz — count the calls

Given the recurrence $T(n) = T(n-1) + T(n-2) + 1$, and the values $T(3) = 4$ and $T(4) = 6$, compute $T(5)$ before checking the answer.

Quiz — answer

Substituting directly into the recurrence: $T(5) = T(4) + T(3) + 1 = 6 + 4 + 1 = 11$. This matches counting the nodes in this lecture's own `fib(5)` recursion tree diagram directly — 11 total function calls appear in that tree, confirming the recurrence against the concretely drawn example rather than trusting the formula in isolation.

A closing note on when memoization is not needed at all

It is worth closing with the boundary of this lecture's technique. If a recursive function's subproblems never genuinely overlap — each distinct call occurs exactly once across the entire recursion — adding memoization contributes pure overhead (a hash table lookup and write on every call) with zero benefit, since there is nothing to reuse. Week 6's merge sort recursion is exactly this case: `merge_sort` on the left half and `merge_sort` on the right half never process the same sub-array twice, so no caching opportunity exists at all. Checking for genuine overlapping subproblems, using this lecture's recursion-tree technique, before reaching for memoization avoids solving a problem that was never actually present in the first place.

Summary

Naive recursive Fibonacci recomputes the same overlapping subproblems repeatedly across its recursion tree, costing $\Theta(\phi^n)$ time, genuinely exponential. Memoization fixes this by caching each distinct subproblem's result in a hash table, directly reusing Week 8's structure, ensuring each distinct subproblem is computed exactly once. This reduces the cost to $O(n)$ — an enormous, not marginal, improvement, achieved with almost no change to the code's structure or apparent complexity. Dynamic programming, as a general technique, applies specifically to problems satisfying two conditions: overlapping subproblems and optimal substructure. Next lecture introduces bottom-up tabulation, an alternative to top-down memoization, and this unit's first genuine optimization problem, the knapsack problem, where recognizing these two conditions is considerably less obvious than it was for Fibonacci.